HW2 BPE tokenizer report - Diyar Husayyan, 214034621

Implementation. All three tokenizers use from-scratch character-up BPE with a mergeable space marker, a single-cross-space bigram cap, byte fallback that never emits UNK, pickle-safe state, and length-faithful decode for the fixed NER aligner. This differs from basic/pre-tokenized BPE by allowing selected cross-word merges while forbidding 3+ word tokens.

Speed (encode). Encoding applies merges greedily by lowest learned rank (tiktoken-style). The naive form rescans the sentence per merge ($O(n^2)$); we replace it with a lazy max-heap over an intrusive doubly-linked list so each merge only re-touches two neighbour pairs - an $O(n \cdot \log n)$ loop that is *byte-identical* to the slow apply-merges-in-order oracle (verified on 13.5k dev lines + edge cases for all tokenizers) yet 10-22x faster on a cold single pass (encode_bench: 1.15-1.24M chars/s here). A per-text cache and weighted-chunk incremental training keep both directions fast; output, efficiency and F1 are unchanged.

Tokenizer choices. tok1 targets Twitter: the balanced domain-1/domain-2 mix at vocab 6000 gave F1 0.4780, +0.043 over the earlier domain-1-only vocab-8000 tokenizer; larger mix vocabs fell sharply, and the structured pre-tokenization experiment was reverted after lowering F1. tok2 targets news with vocab 12000: an equal-axis sweep found F1 0.9606, slightly faster cold encode than vocab 16000, and only a modest efficiency cost. tok3 handles the hidden domain with the balanced deduped mix, vocab 6000, and byte fallback; visible-domain checks are d1=0.4780, d2=0.9621, OOV stress 15/15, UNK=0.

model/split	vocab	dev F1	tok/char	tok/sent	cold tok/s
tok1 domain_1	6000	0.4780	0.307	22.52	269836
tok2 domain_2	12000	0.9606	0.241	21.77	187317
tok3 domain_1	6000	0.4780	0.307	22.52	267020
tok3 domain_2	6000	0.9621	0.287	25.93	270166

Training-set bigrams. These are full-file counts of adjacent whitespace-delimited two-word sequences, as required by the assignment.

tok	train file	top 5 two-word sequences	least 5 two-word sequences
tok1	data\domain_1_train.txt	in the (34889); going to (29182); for the (27436); on the (23322); to be (23173)	! !Damn (1); ! #Apprentice (1); ! #Big (1); ! #FB (1); ! #FrenchMcflyTeam (1)
tok2	data\domain_2_train.txt	of the (15662); in the (12929); to the (6643); on the (5665); to be (5548)	!! Shot (1); !!! It’s (1); !&*@#%! Watch (1); !@#$-ing Kidding (1); !T Books (1)
tok3	data\tok3_mix_d1sample_d2all_seed42.txt	in the (21479); of the (20299); to the (12293); for the (11919); on the (11441)	! #FB (1); ! #MetaKong (1); ! #Myspace-y (1); ! #SMTB (1); ! #familyforce5 (1)

Cross-word bigrams (studied tradeoff). Each shipped tokenizer deliberately encodes exactly one checker-defined real bigram token: comma+quote followed by the space-marker form of "the". We also implemented a controlled, reproducible collocation phase restricted to corpus-wide whole alphabetic word pairs. On tok1, K=40 alpha-word collocations cut tokens/sentence by only 0.08% (22.5193 -> 22.5008); examples: I am, How To, I have, Donald Trump, This is. The pairs are cleaner than punctuation mining but still include named-entity/context joins, and the best-case efficiency gain is too small to risk word-level NER F1. Therefore the submitted code ships K=0: one mandatory bigram, zero trigrams, zero UNK.

Hidden-domain and model-2 discussion. tok3 now favors the equal-axis knee: vocab 6000 recovers the best visible F1/speed signal while keeping exact reconstruction and no UNK on stress cases; the efficiency cost versus vocab 8000 is smaller than the F1 gain. tok2's high news F1 reflects two concrete properties of the news corpus: formal spelling/punctuation makes subword boundaries stable, and repeated named-entity/title patterns let vocab 12000 compress frequent fragments without fragmenting labels. Thus tok2 improves F1/speed over vocab 16000 while retaining most of its compression.